

操作系统内核阶段性设计文档

Comix

面向内核设计评审与阶段提交

2026 年 6 月

目录

1. 摘要
2. 系统概述
3. 代码结构与文档组织
4. 架构抽象与启动流程
5. 内存管理模块
6. 任务与调度模块
7. 文件系统与 VFS 模块
8. IPC 与信号模块
9. 同步与并发控制
10. 网络模块
11. Trap 与异常处理
12. 系统调用边界
13. 设备与存储路径
14. 日志与诊断
15. 验证情况与阶段展望

摘要

Comix 是一个使用 Rust 编写的操作系统内核。本阶段文档总结当前内核在架构抽象、启动流程、内存管理、任务调度、Trap 与系统调用、文件系统、IPC、同步、网络、设备和日志诊断上的设计状态。文档重点不是复述函数级 API，而是说明系统分层、关键流程、约束和仍需收敛的限制。

当前正式设计文档位于 `document/`。该目录采用“细项主导航 + 简洁页面”的方式组织：`SUMMARY.md` 保留可直接跳转的子系统细项，页面正文强调设计边界、生命周期和不变量；函数签名、字段含义、错误分支等实现细节则由 `rustdoc` 和源码注释承载。

本阶段的核心目标是让文档与最新代码同步，并形成一份可用于阶段提交的汇总材料。本文参考传统操作系统内核设计文档的封面、目录和分章方式，但内容只基于 Comix 当前文档和源码结构，不把历史方案或未接入实现写成当前能力。

1.1 当前阶段重点

领域	阶段状态
架构支持	RISC-V64 和 LoongArch64 通过统一 <code>arch</code> 门面接入通用内核；RISC-V 已接入多核启动、IPI、定时器和外部中断路径，LoongArch64 当前以单核 <code>bringup</code> 和用户态运行为主。
内存管理	提供强类型地址抽象、物理帧分配、 <code>talc</code> 全局堆、页表抽象和 <code>MemorySpace</code> 地址空间管理，支持 ELF 加载、 <code>mmap</code> 、 <code>fork</code> 克隆和 SysV 共享内存映射。
任务系统	用统一 <code>Task</code> 模型承载进程、线程和内核线程；调度器维护运行状态与 <code>run queue</code> ，任务管理器维护 <code>tid</code> 、父子关系和退出状态。
文件系统	VFS 提供路径、 <code>dentry</code> 、 <code>inode</code> 、 <code>file</code> 和 <code>fd table</code> 抽象；具体 FS 包括 <code>ext4</code> 、VFAT/FAT、 <code>tmpfs</code> 、 <code>procfs</code> 、 <code>sysfs</code> 和 <code>simple_fs fallback</code> 。
IPC 与网络	IPC 支持 <code>pipe</code> 、内核消息队列、SysV shared memory 和 <code>signal</code> ；网络通过 <code>NetDevice</code> 、接口控制面、 <code>smoltcp runtime</code> 、VFS socket 文件和 <code>syscall</code> 层连接。

系统概述

Comix 采用分层内核结构。架构层负责 CPU、trap、页表和平台 hook；通用内核层负责启动、任务、调度、系统调用和 CPU 状态；基础设施层提供内存管理、同步、VFS、具体文件系统、IPC、网络与设备抽象。用户态通过系统调用进入内核，系统调用层完成 ABI 边界处理后转发到各子系统。

用户态程序

```
-> trap 和 syscall frame
-> syscall dispatch
-> task / mm / vfs / fs / ipc / net / signal
-> arch / device / platform
```

2.1 分层原则

- 架构差异被限制在 `os/src/arch/`，通用内核通过 trait、类型别名和包装函数访问架构能力。
- 系统调用层只处理 ABI、用户指针、fd 查找和 errno 映射，不承载跨子系统共享状态。
- VFS 负责统一文件命名空间和打开文件模型，具体文件系统只实现 `FileSystem` 与 `Inode` 边界。
- 任务生命周期、调度状态、地址空间、fd table、signal 和 shm attachment 分属不同模块，通过清晰的资源所有权约束协作。

2.2 当前限制

Comix 当前不是完整 Linux 内核兼容实现。部分 syscall 为兼容测试提供最小语义；LoongArch64 的 SMP、IPI 和外部设备中断路径尚未与 RISC-V 对齐；内存管理中的 fork 深拷贝尚未收敛为完整 COW；文件系统、网络 and signal 的高级语义仍在持续完善。

代码结构与文档组织

本阶段文档同步后，正式设计文档位于 `document/`，源码位于 `os/src/`。文档不再维护完整 API 清单，而是把设计页和 rustdoc 分工固定下来。

```
comix-1/  
  document/  
    SUMMARY.md  
    book.toml  
    arch/ kernel/ mm/ sync/ vfs/ fs/ ipc/ net/ syscall/  
    reports/  
  os/src/  
    arch/ kernel/ mm/ sync/ vfs/ fs/ ipc/ net/ device/ log/
```

3.1 文档分层

层级	职责
<code>document/</code>	维护设计、边界、关键流程、不变量、已知限制和源码索引。
rustdoc	维护公共类型、函数、错误条件、调用约束和局部示例。
源码注释	解释 unsafe 依据、锁顺序、架构细节和非显而易见的局部逻辑。

3.2 mdBook 与阶段 PDF

`document/book.toml` 使当前目录布局可以直接执行 `mdbook build document`。本 PDF 的源文件位于 `document/reports/stage-report.html`，使用浏览器打印引擎导出为 A4 PDF，适合阶段提交和评审传阅。

架构抽象与启动流程

架构层的核心职责是把 CPU 和平台差异约束在 `arch` 目录内部。`os/src/arch/mod.rs` 是架构层公开门面，通用内核通过 `ArchImpl`、`PlatformImpl` 和统一 `TrapFrame` 类型访问架构能力。

4.1 当前架构状态

- RISC-V64: 多核启动、软件中断 IPI、定时器中断和外部中断路径已经接入通用调度。
- LoongArch64: 当前以单核 bringup 和用户态运行为主，保留同名接口以复用通用内核代码。
- Mock 架构: 用于宿主测试构建核心模块。

4.2 公共启动流程

启动主线已经从各架构目录收敛到 `os/src/kernel/boot.rs`。架构入口只完成本架构必须先做的 CPU 指针、CSR/FPU 等早期状态，然后调用公共启动流。

```
arch::boot::main
-> before_clear_bss
-> clear_bss
-> mm::init
-> switch to kernel address space
-> init_boot_trap
-> platform::init
-> time::init
-> timer::init
-> create CPU0 idle task
-> trap::init
-> rest_init
-> enable interrupts
-> idle_loop
```

`rest_init` 创建 PID 1，PID 1 初始仍以内核任务形式运行，随后初始化 rootfs 和网络并通过 `kernel_execve("/sbin/init")` 进入用户态 `init`。RISC-V 从核通过 SBI HSM 进入 secondary 流程，LoongArch64 当前保持单核语义。

内存管理模块

MM 子系统负责物理帧分配、内核堆、页表抽象和进程地址空间管理。架构无关策略位于 `os/src/mm/`，页表格式、TLB 刷新和地址转换位于 `os/src/arch/{riscv, loongarch}/mm/`。

5.1 模块组成

- 地址抽象：PA、VA、UA、Ppn、Vpn 等强类型包装，减少地址空间误用。
- 物理帧分配：全局 `SpinLock<FrameAllocator>` 保护，已分配帧通过 RAII tracker 回收。
- 全局堆：使用 `tal::Talck<RawSpinLock, ClaimOnOom>`。
- 页表抽象：通用 PTE flag 由架构后端转换成实际硬件 PTE。
- 地址空间：`MemorySpace` 维护页表与 `MappingArea` 列表，支持 `brk`、`mmap`、`munmap`、`mprotect`、ELF 加载、fork 克隆、文件映射和 SysV shared memory 映射。

5.2 设计约束

当前页表路径只启用 4K 页。用户页表包含用户私有映射和内核共享映射，通过 PTE 用户访问位隔离权限。`MappingArea` 是 VMA 级别的元数据和帧所有权边界，页表只保存硬件可见映射，不记录映射来源。

5.3 已知限制

- `MemorySpace::areas` 仍是线性 `Vec`，区域查找和重叠检测是线性扫描。
- fork 对 `Framed` 区域做深拷贝，尚不是完整 COW。
- LoongArch TLB 批处理上下文当前是本地刷新占位实现。
- 大页映射尚未作为正式能力暴露。

任务与调度模块

任务子系统定义内核运行单元、生命周期和阻塞/唤醒边界。Comix 使用统一 `Task` 表示进程、线程和内核线程，所有任务通过 `SharedTask = Arc<SpinLock<Task>>` 在调度器、任务管理器、wait queue、信号和文件系统之间传递。

6.1 任务模型

- 进程是 `pid == tid` 的线程组 leader，线程与 leader 共享部分资源。
- 内核线程没有用户地址空间，但和用户任务共用第一次调度入口 `forkret`。
- 每个任务具有独立内核栈、`TrapFrame` 和调度 `Context`。
- `execve` 替换用户地址空间，重建用户栈和 `TrapFrame`。

6.2 生命周期

```
create
-> Running and queued
-> scheduled on CPU
-> running in kernel or user mode
-> sleep on event
-> wake and queued again
-> exit to Zombie
-> parent wait or release removes global reference
```

当前 `Running` 同时表示正在 CPU 上运行或可运行并等待队列调度。任务管理器维护全局 `tid` 映射、父子关系和退出状态；调度器负责运行状态转换和 `run queue` 维护。

6.3 并发约束

- `Task` 内部由 `SpinLock` 保护，不应长期持有任务锁后调用可能唤醒或调度的路径。
- 同一任务不能被两个 CPU 同时调度运行。
- 多核唤醒必须幂等，已经是 `Running`、`Zombie` 或 `Stopped` 的任务不能重复入队。

文件系统与 VFS 模块

VFS 是 Comix 的统一文件命名空间和打开文件模型。它把路径、文件描述符、设备文件和具体文件系统实现分离，使 ext4、tmpfs、procfs、sysfs、VFAT 以及设备节点都能通过同一组抽象接入。

7.1 VFS 核心对象

对象	职责
File	打开文件会话，保存 offset、open flags 和具体读写语义。
Inode	文件系统对象，提供随机访问、目录操作和元数据。
Dentry	路径缓存节点，连接名字、父子关系和 inode。
MountTable	维护全局挂载点栈，路径解析自动跨越挂载边界。
FDTable	进程可见 fd 空间，fd 指向 Arc<dyn File>。

7.2 当前文件系统实现

- ext4: 默认 rootfs 候选，支持持久化读写。
- VFAT/FAT: 用于 mount/umount 兼容路径和测试分区。
- tmpfs: 提供内存临时文件树。
- procfs: 生成进程和系统状态快照。
- sysfs: 将设备注册表映射为 /sys 视图。
- simple_fs: 编译期嵌入的只读 fallback rootfs。

7.3 rootfs 探测流程

```
device discovery
-> list block disks and partitions
-> sort partitions before whole disks
-> try ext4 open
-> mount temporarily at /
-> accept only if /bin/sh or /bin/ash exists
-> create common mount dirs
-> mount procfs, sysfs, tmpfs and create /dev nodes
```

IPC 与信号模块

IPC 子系统由一组和 VFS、Task、MemorySpace、syscall 层协作的机制组成，负责字节流、离散消息、共享页和异步事件。

8.1 IPC 机制

- Pipe：底层是环形字节缓冲区，对外通过 VFS PipeFile 暴露为 fd。
- Message：内核内消息队列，以消息类型和 payload 为边界，通过等待队列提供阻塞收发。
- SysV shared memory：全局 segment registry 管理 shmid/key，syscall 层将 segment 映射进当前 MemorySpace。
- Signal：任务私有 pending 与线程组共享 pending 共同决定返回用户态前的投递行为。

8.2 生命周期约束

IPC 对象通常由 Arc 持有，生命周期由 fd table、task attachment table 或全局 registry 共同决定。等待路径必须避免在持有长生命周期锁时睡眠。进程退出或 exec 时由 task 清理路径关闭 fd、分离 shm、释放地址空间。

8.3 当前限制

- MessageQueue 当前是内核内队列，没有完整 msgget/msgsnd/msgrcv/msgctl ABI。
- Signal 尚未完整实现 SA_RESTART 和实时信号队列。
- Pipe 的阻塞和 poll 语义主要在 VFS PipeFile 中体现。

同步与并发控制

`os/src/sync` 提供当前内核使用的同步原语。文档只描述现行实现和设计边界，不把未接入源码的历史方案当作可用 API。当前 `ticket_lock.rs` 和 `sleep_lock.rs` 不存在于源码中，对应文档页仅作为历史状态保留。

9.1 当前同步原语

原语	用途
<code>RawSpinLock</code>	<code>AtomicBool</code> 互斥，进入时持有 <code>IntrGuard</code> ，也适配 <code>lock_api::RawMutex</code> 。
<code>SpinLock<T></code>	短临界区互斥，适合极短共享数据修改。
<code>RwLock<T></code>	短临界区的多读单写访问，持 <code>guard</code> 期间禁用本 CPU 中断。
<code>Mutex<T></code>	任务上下文互斥，竞争时进入 <code>wait queue</code> 并 <code>yield</code> 。
<code>IntrGuard</code>	禁用和恢复本 CPU 中断，支持 per-CPU 嵌套深度。
<code>PreemptGuard</code> / <code>PerCpu<T></code>	防止访问当前 CPU 数据时任务迁移。

9.2 并发约束

- 自旋锁类临界区必须短，不能主动 `sleep` 或长期等待调度。
- `Mutex<T>` 可能调用调度相关路径，只适合任务上下文。
- `PerCpu<T>::get_mut()` 要求调用方保证访问期间不会迁移。

网络模块

网络模块连接 `NetDevice`、接口控制面、smoltcp runtime、VFS socket 文件和网络 syscall。当前仍是单 active runtime 设计，但边界已经集中到 `NetworkStack` 门面。

10.1 当前能力

- 真实网卡通过 `register_net_device()` 注册为 `NetworkInterface`。
- 默认配置保证 `lo` 存在，loopback-only 场景使用 `127.0.0.1/8` 且没有默认网关。
- `AF_INET` TCP/UDP socket 由 smoltcp 驱动，状态集中在 `NetworkStack`。
- `AF_UNIX` socket 是内核本地 IPC transport，不进入 smoltcp 数据路径。
- `SocketFile` 和 `UnixSocketFile` 实现 VFS `File`，通过 fd table 暴露给用户态。

10.2 分层边界

设备层只提供收发帧能力；接口层保存控制面配置；协议栈运行时由 `NetworkStack` 持有和推进；syscall 层处理用户 ABI、fd table、sockaddr 和 errno。当前非目标包括多 active interface runtime 和把 `AF_UNIX` 混入 smoltcp 数据路径。

Trap 与异常处理

Trap 路径连接硬件异常、中断、系统调用和调度器。RISC-V 与 LoongArch 都提供 `arch::trap` 模块，对通用内核暴露初始化、恢复、信号 trampoline 和 `TrapFrame` 操作。架构汇编入口保存现场后进入 Rust handler，handler 分派 syscall、timer、IPI 或设备中断，最终恢复当前任务的 `TrapFrame`。

11.1 通用 Trap 流程

```
hardware trap
-> arch trap_entry
-> save TrapFrame
-> Rust trap_handler
-> syscall/timer/IPI/device/exception dispatch
-> maybe schedule
-> restore current task TrapFrame
-> sret/ertn
```

11.2 架构状态

- RISC-V 路径已覆盖 syscall、timer、software interrupt IPI 和 external interrupt。
- LoongArch 路径已覆盖 syscall、timer、TLB refill 入口安装和基本恢复。
- 系统调用会调整返回 PC，然后把 `TrapFrame` 交给 syscall dispatcher。
- 时钟中断推进全局 ticks、timer queue 和 interval timer，必要时触发调度。

11.3 约束与限制

- trap handler 入口时中断通常已关闭，返回必须通过架构 restore 恢复特权状态和中断状态。
- trap handler 中不要执行可能阻塞或长期持锁的工作。
- 如果 handler 中发生任务切换，返回必须读取当前任务的 `trap_frame_ptr`。
- LoongArch 外部中断和 IPI 尚未接入完整分派，用户态未知异常仍偏 bringup 诊断。

系统调用边界

Syscall 层是用户态 ABI 到内核子系统的边界。`dispatch_syscall()` 按 syscall number 分发到 `sys_*` 包装函数；`SyscallFrame` trait 抽象寄存器读取和返回值写回；`impl_syscall!` 宏负责从 frame 提取参数、调用实际实现并写回返回值。

12.1 Dispatch 职责

1. 读取 syscall id 和参数用于调试日志。
2. 根据 `numbers.rs` 中的常量选择对应 `sys_*` wrapper。
3. 对未知号码写回 `-ENOSYS`。

dispatch 不直接读取用户内存，不操作 fd table，也不进入 VFS 或协议栈内部状态。真正实现按领域拆到 fs、io、task、mm、signal、ipc、network、sys、cred 等模块。

12.2 用户指针原则

- 用户指针只能在 syscall 边界或明确的用户缓冲工具中解引用。
- 不把用户指针保存到内核对象中。
- 持有协议栈、VFS、MemorySpace 等关键锁时避免长时间复制用户缓冲区。
- 复制失败返回 `-EFAULT` 或对应子系统错误。

设备与存储路径

设备层为 Comix 提供驱动注册、设备树探测、VirtIO 传输、块设备、网络、控制台和 RTC 等基础能力。对当前阶段最关键的是存储路径：块设备和分区被枚举为 `vda`、`vda1`、`vda2` 等名字，上层 FS 再从候选中探测 rootfs 或挂载 VFAT 测试分区。

13.1 驱动模型

- 所有驱动实现 `Driver`，并可按类型向下暴露 `BlockDriver`、`NetDevice`、`RtcDriver`、`SerialDriver`。
- 全局注册表包括 `DRIVERS`、`BLK_DRIVERS`、`RTC_DRIVERS` 和 `SERIAL_DRIVERS`。
- 设备树初始化先处理中断控制器，再处理普通设备；VirtIO MMIO 是主要设备传输路径。
- `sysfs` 通过设备注册表构建 `/sys/class/*`，FS 初始化通过同一设备列表创建 `/dev` 节点。

13.2 存储关键流程

```
device_tree init
-> virtio mmio probe
-> virtio block init
-> BLK_DRIVERS push whole disk
-> sysfs device_registry list_block_devices
-> discover partitions
-> vda, vda1, vda2 ...
```

`BLK_DRIVERS` 只保存整盘驱动，分区设备在 `list_block_devices` 时根据分区表动态包装为逻辑块设备。`rootfs` 探测不依赖固定 `virtio` 枚举顺序，而是优先尝试分区并通过文件内容判断。

13.3 已知限制

- 设备热插拔后 `/dev` 和 `/sys` 不是自动增量更新模型。
- 分区解析支持 primary MBR 和基础 GPT 条目，不覆盖所有复杂分区格式。
- 分区发现默认假设 512 字节扇区。

日志与诊断

Log 子系统提供内核级分级日志、固定大小环形缓冲、控制台即时输出和 `syslog` 读取接口。当前设计偏向早期可用和低依赖，确保关键诊断信息可以在不同启动阶段输出。

14.1 日志路径

1. 调用 `pr_info!` 等宏。
2. 宏先检查 global level，被过滤的日志不进入格式化。
3. `LogCore` 收集上下文并构造固定大小 `LogEntry`。
4. 条目写入多生产者单消费者环形缓冲。
5. 若级别达到 console level，直接格式化输出到控制台。
6. 用户态通过 `syslog` 读取格式化后的日志文本。

14.2 设计边界

- 全局 `LogCore` 使用 `const fn` 静态初始化，无需运行时 init。
- `print!` 和 `println!` 保持控制台输出，同时以 Info 级别写入日志缓冲。
- panic/trap 关键路径可使用 `console::emergency_print()` 绕开常规日志路径。
- 日志系统不承担持久化、审计、tracing 或结构化事件系统职责。

14.3 已知限制

- `syslog` 权限检查当前是兼容 stub。
- 缓冲区大小固定为编译期常量，高日志量场景会覆盖旧日志。
- ANSI 颜色码会进入 `syslog` 格式化输出。

验证情况与阶段展望

本阶段文档更新完成后，已对正式文档和 rustdoc 进行基础验证。验证重点是目录可构建、链接可达、旧路径不再作为当前实现引用，以及 rustdoc 不因文档注释问题失败。

15.1 已执行验证

验证项	结果
<code>mdbook build document</code>	通过，输出目录为 <code>target/mdbook-document</code> 。
<code>cargo doc --no-deps --target riscv64gc-unknown-none-elf</code>	通过，仅存在已有的第三方或构建 warning。
<code>git diff --check --document os/src</code>	通过。
<code>SUMMARY.md</code> 链接检查	通过。
旧路径和过期文案检查	未发现 <code>os/src/fs/fat32</code> 、 <code>os/src/sync/ticket_lock.rs</code> 、 <code>os/src/sync/sleep_lock.rs</code> 等作为当前实现路径残留。

15.2 阶段展望

- 内存管理：收敛 COW、区域索引结构和跨架构 TLB shutdown 同步策略。
- 任务系统：拆分调度热字段和进程资源字段，完善抢占计数与优先级模型。
- 架构层：推进 LoongArch64 SMP、IPI 和外部中断路径与 RISC-V 对齐。
- 文件系统：完善 dentry 失效、mount flags 执行语义和动态文件树一致性。
- 信号与 syscall：完善 `SA_RESTART`、实时信号队列和更多 Linux ABI 语义。
- 网络：从单 active runtime 继续向更完整的接口和 socket 语义推进。

本文是阶段性设计汇总，详细子系统说明以 `document/` 下的 mdBook 文档为准，函数级行为以 rustdoc 和源码注释为准。